

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO1 ASSESSMENT TOOL 1 – Case Study Analysis

Course Code:	CSA10 / CSA1063	Course Title:	Software Engineering
CO Assessed:	CO1	Assessment:	Assessment Tool 1 – Case Study Analysis
Weightage:	20%	Total Marks:	25
CO1 Statement:	Apply advanced methodologies and agile project management techniques to manage software projects effectively		
Student Name:	YANATI HEMA SAI LAHARI	Reg. No.:	192311446
Date:	14-08-2026	Duration:	60 minutes

Code of Conduct

I, YANATI HEMA SAI LAHARI / 192311446 No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: YANATI HEMA SAI LAHARI / 192311446

Annexure A – Case Study Analysis

Instructions to Students:

Each student will be allotted **one problem statement**. Analyze the assigned problem systematically by following the steps given below. Your report should demonstrate your understanding of software engineering concepts and your ability to design an appropriate software solution.

Based on your allotted problem statement, prepare a report by answering the following questions:

1. Understand the Problem Statement

- Explain the problem in your own words.
- Identify the objectives of the proposed system.
- State the scope of the problem.

2. Identify Key Stakeholders and Challenges

- Identify all the stakeholders involved in the system.
- Explain the roles and responsibilities of each stakeholder.
- Discuss the major challenges and issues faced by the stakeholders.

3. Analyze the Current Approach

- Describe the existing system or current process.
- Identify its strengths and limitations.
- Analyze the problems or gaps that need to be addressed.

4. Propose a Suitable Solution Using Software Engineering Principles

- Design a software-based solution for the given problem.
- Describe the major functional modules of the proposed system.
- Identify the functional and non-functional requirements.
- Explain how software engineering principles such as modularity, scalability, maintainability, reliability, usability, and security are incorporated into your solution.

5. Justify the Chosen Methodology/Framework

- Select an appropriate Software Development Life Cycle (SDLC) model or software development framework (e.g., Agile, Scrum, Waterfall, Spiral, DevOps).
- Justify why the selected methodology/framework is suitable for the proposed system.
- Explain how it supports successful project development and maintenance.

6. Discuss Expected Outcomes and Limitations

- Describe the expected benefits of the proposed solution.
- Explain how the solution addresses the identified challenges.
- Discuss potential risks, implementation challenges, and limitations.
- Suggest possible future enhancements.

Rubrics for Calculation

Criteria	Excellent (5 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
Problem Understanding (30%)	Clearly explains the problem, objectives, and scope with complete understanding.	Explains the problem with minor omissions.	Basic understanding with limited explanation.	Poor understanding; objectives and scope are unclear or missing.
Stakeholder & Challenge Analysis (25%)	Identifies all stakeholders, their roles, and challenges with clear analysis.	Identifies most stakeholders and challenges with adequate analysis.	Identifies only a few stakeholders or challenges.	Stakeholders and system analysis are incomplete or inaccurate.
Current System Analysis (20%)	Thoroughly analyzes the existing system, strengths, weaknesses, and identifies all gaps.	Good analysis with most strengths and weaknesses identified.	Limited analysis with partial identification of issues.	Proposed solution is unclear or inappropriate; requirements are largely missing.
Proposed Software Solution & Software Engineering Principles (15%)	Proposes an innovative, feasible solution applying appropriate software engineering principles (modularity, scalability, maintainability, security, etc.).	Suitable solution with good application of software engineering principles.	Basic solution with limited application of software engineering principles.	Methodology is inappropriate, poorly justified, or not discussed.
Methodology/ Framework Justification	Selects the most suitable SDLC/model/fram	Suitable methodology selected with	Methodology selected but justification is	Outcomes, limitations, or future enhancements

(10%)	ework and provides strong technical justification.	adequate justification.	weak.	are missing; report lacks organization and clarity.

Criteria	Mark
Problem Understanding (30%)	/5
Stakeholder & Challenge Analysis (25%)	/5
Current System Analysis (20%)	/5
Proposed Software Solution & Software Engineering Principles (15%)	/5
Methodology/ Framework Justification (10%)	/5
TOTAL	/25

1. Understanding the Problem Statement

1.1 Problem in Own Words

Temples across India collect a very large volume of donations every year — cash offerings (hundials), online transfers, gold/silver or kind donations, and sponsorships for poojas, festivals and annadhanam (free meals). At the same time, temple administrations organise a continuous calendar of events: daily rituals, weekly poojas, annual festivals, weddings, and cultural programs, all of which require halls, priests, volunteers and other resources to be booked without conflict. In most temples today, both of these activities — donation handling and event management — are still run manually using paper registers, physical receipt books, handwritten ledgers and word-of-mouth coordination among staff. This creates delays in issuing receipts, makes it difficult to reconcile cash against records, leaves the trust board with no real-time visibility into finances, and frequently results in double-booking of halls or priests for overlapping events. The Temple Donation and Event Management System (TDEMS) is proposed as an integrated software solution that digitises both donation collection and event scheduling, so that devotees, volunteers, accountants and trustees all work from one accurate, transparent, real-time system.

1.2 Objectives of the Proposed System

- Provide a single digital platform to record, track and report all donations — online and offline, cash and in-kind.
- Instantly generate donation receipts and 80G tax-exemption certificates for every contribution.
- Give donors a personal history of their contributions and easy repeat-donation options.
- Enable trustees/administrators to schedule events and book halls, priests and other resources with automatic conflict detection.
- Coordinate volunteers and staff through task assignment and notifications instead of ad-hoc phone calls.
- Give the trust board real-time dashboards and exportable reports for internal review and statutory/Endowment-Department compliance.
- Integrate secure, PCI-DSS-compliant payment gateways to widen the donor base, including NRIs and remote devotees.
- Ensure the system is secure, auditable and able to scale during high-traffic festival periods.

1.3 Scope of the Problem

In scope: online and offline donation entry and receipting; donor profile and history management; event creation, hall/priest/resource booking with conflict checking; volunteer/staff task management; role-based dashboards and financial/audit reporting; payment gateway integration; SMS/email notifications; and role-based user access (Donor, Volunteer, Event Coordinator, Accountant, Admin/Trustee).

Out of scope (for this phase): temple inventory and prasadam supply-chain management, physical security/CCTV systems, full HR/payroll processing for temple staff, and management of unrelated temple properties or land records. These may be considered as future extensions once the core donation-and-event platform is stable.

2. Identify Key Stakeholders and Challenges

The system touches a wide range of people, each with distinct needs and pain points under the current manual process.

Stakeholder	Role / Responsibility	Key Challenges Faced
Devotees / Donors	Contribute money or goods; view receipts and donation history	No instant receipt; no visibility into how funds are used; inconvenient donation modes for those living abroad or far from the temple.

Stakeholder	Role / Responsibility	Key Challenges Faced
Temple Administration / Trust Board	Overall governance, fund utilisation decisions, statutory compliance	Manual bookkeeping makes audits slow and error-prone; limited real-time visibility into cash flow and event status.
Accountant / Finance Staff	Record donations, reconcile cash vs online, prepare reports and certificates	Manual reconciliation of cash counters with bank statements; time-consuming manual generation of 80G certificates; high risk of data-entry errors.
Event Coordinators / Priests	Plan and conduct festivals, poojas, weddings; request halls and resources	Scheduling conflicts and double-booking of halls/priests; miscommunication over dates and resource availability.
Volunteers / Staff	Execute event-day tasks: crowd management, prasadam distribution, ushering	No centralised task list; coordination happens informally over phone/WhatsApp, leading to missed or duplicated tasks.
Regulatory Bodies (Endowment Dept., Income Tax)	Audit and compliance oversight of temple finances	Manual records make timely, accurate statutory reporting difficult; higher risk of non-compliance.
IT/System Administrator	Maintain the system, manage users, ensure security and backups	No existing digital infrastructure to build on; must design access control and safeguard sensitive donor/financial data.

3. Analyze the Current Approach

3.1 Existing System / Process

Today, most temples run on a paper-based process: donations are entered by hand into counter registers or receipt books, festival/hall bookings are noted in a physical diary or coordinated informally over phone calls and WhatsApp groups, and monthly accounts are compiled at the end of each month, often re-typed into a spreadsheet by the accountant for the trust board. Very small or occasional online donations may be accepted through a bank account, but they are reconciled with the physical register manually.

3.2 Strengths of the Current Approach

- Low upfront cost — no software licensing, hosting or training investment.
- Familiar and easy to use for long-serving staff and elderly volunteers with limited digital literacy.
- No dependency on internet connectivity or power for day-to-day operation.

3.3 Limitations of the Current Approach

- Records are error-prone and time-consuming to reconcile, especially during high-volume festival days.
- Physical receipts and registers can be lost, damaged, or manipulated, with no backup copy.
- No real-time visibility for trustees into total donations, fund utilisation, or event status.
- Halls, priests and resources are booked informally, causing frequent double-booking and last-minute conflicts.
- Donors outside the local area (especially NRIs) have limited, inconvenient ways to contribute.
- Manual generation of 80G certificates and statutory reports is slow and increases the risk of compliance errors.
- Cash handling with minimal automated cross-checking increases the risk of misappropriation or fraud.

3.4 Gaps to Be Addressed

The analysis of the current approach points to five core gaps that the proposed system must close: (1) digitisation of donation capture and receipting, (2) integration of donation records with event/resource scheduling so both are managed

from one platform, (3) a secure online payment channel to widen donor reach, (4) real-time, role-based reporting for transparency and compliance, and (5) resilience and data security so records can never be permanently lost, altered without a trace, or exposed to unauthorised access.

4. Propose a Suitable Solution Using Software Engineering Principles

4.1 Overview of the Proposed Solution

The Temple Donation and Event Management System (TDEMS) is proposed as a modular, web- and mobile-accessible platform built on a layered architecture — a presentation layer for devotees, staff and trustees; a business-logic layer of independent functional modules; an external integration layer for payment and messaging gateways; and a secure data layer with encrypted storage and backups. Figure 1 illustrates the proposed architecture.

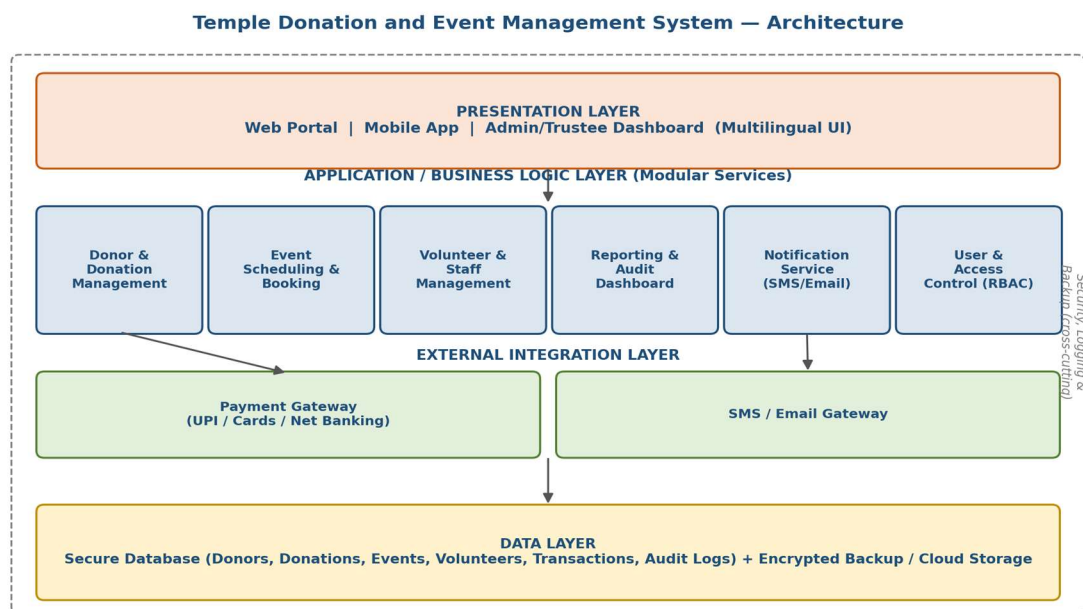


Figure 1: Proposed layered architecture of the Temple Donation and Event Management System

4.2 Major Functional Modules

- **Donor & Donation Management:** records donor details, captures both online and offline donations, and links each donation to an auto-generated receipt.
- **Payment Gateway Integration:** connects to UPI, card and net-banking gateways for secure online contributions, and auto-reconciles them with the ledger.
- **Event Scheduling & Booking:** lets administrators create events, book halls/priests/resources, and automatically flags scheduling conflicts.
- **Volunteer & Staff Management:** assigns and tracks volunteer tasks for each event, with attendance and reminder notifications.
- **Reporting & Audit Dashboard:** gives trustees and accountants real-time dashboards, exportable financial reports and a tamper-evident audit trail.
- **Notification Service:** sends automated SMS/email alerts for receipts, event reminders and task assignments.

- **User & Access Control (RBAC):** enforces role-based permissions so each user (Donor, Volunteer, Accountant, Event Coordinator, Admin/Trustee) only sees and does what their role allows.

Figure 2 shows how these modules are exercised by the different actors as a use-case view of the system.

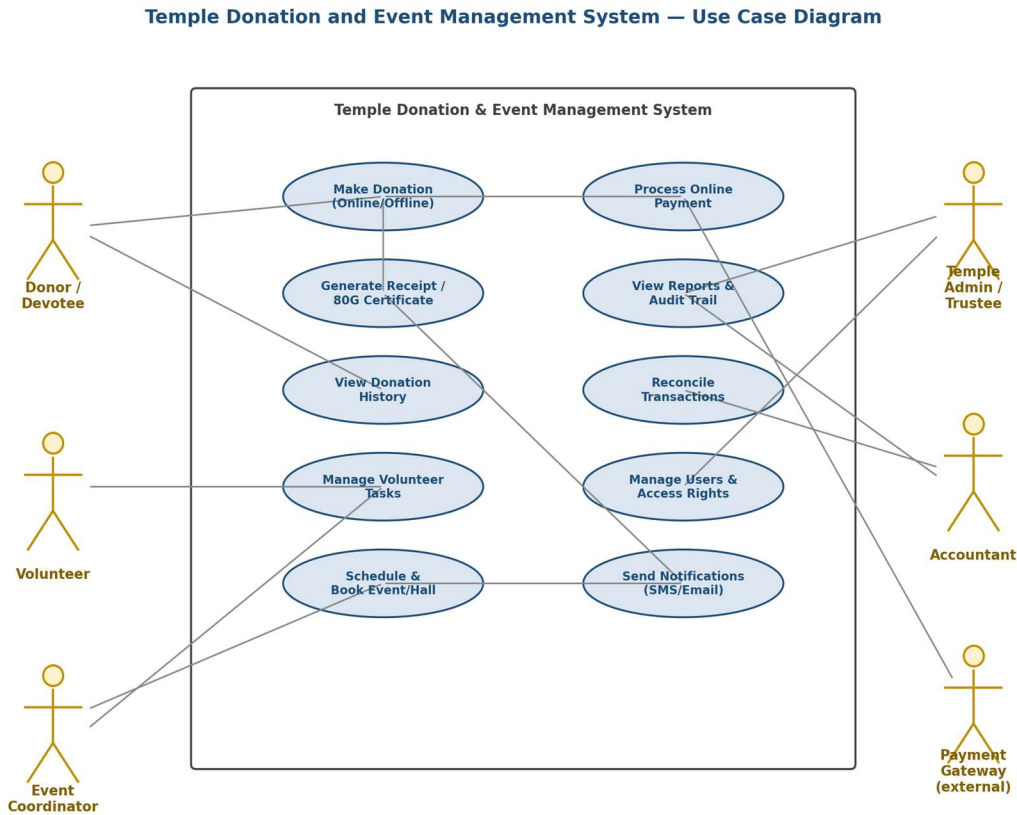


Figure 2: Use-case view of the Temple Donation and Event Management System

4.3 Functional Requirements

ID	Functional Requirement
FR-1	The system shall allow devotees to make donations online (UPI/card/net-banking) or record offline (cash/kind) donations at the counter.
FR-2	The system shall auto-generate a digital receipt and, where applicable, an 80G tax-exemption certificate for every donation.
FR-3	The system shall maintain a donor profile with complete donation history, searchable by donor or date range.
FR-4	The system shall allow administrators/trustees to create events (festivals, poojas, weddings) and book halls, priests and resources, with automatic conflict detection for double-booking.
FR-5	The system shall allow volunteer coordinators to assign, track and update tasks for volunteers linked to specific events.
FR-6	The system shall provide role-based dashboards showing real-time donation totals, event calendars and audit logs.
FR-7	The system shall send automated SMS/email notifications for receipts, event reminders and task assignments.

ID	Functional Requirement
FR-8	The system shall allow the accountant to reconcile online and offline donations and export financial reports (daily/monthly/annual, and Endowment-Department formats).
FR-9	The system shall enforce role-based access control (Donor, Volunteer, Accountant, Event Coordinator, Admin/Trustee).

4.4 Non-Functional Requirements

Quality Attribute	Requirement
Performance	The system shall support at least 500 concurrent transactions during peak festival periods without noticeable slowdown.
Scalability	The architecture shall scale horizontally (cloud/microservice deployment) to absorb seasonal traffic spikes.
Reliability	The system shall maintain 99.5%+ uptime with automated daily backups and disaster recovery.
Usability	The interface shall be simple and multilingual (e.g., Tamil, Telugu, Hindi, English) for users of varying digital literacy, including elderly staff and devotees.
Security	All financial and personal data shall be encrypted in transit and at rest; payments shall follow PCI-DSS-compliant gateways; every action shall be logged in an immutable audit trail.
Maintainability	The system shall be built as loosely coupled modules with documented APIs to simplify future updates and bug fixes.
Availability	Donation and receipt services shall remain available 24/7, including during high-traffic festival days.

4.5 How Software Engineering Principles Are Incorporated

- **Modularity:** Each functional area (donations, events, volunteers, reporting, notifications, access control) is built as an independent module with a well-defined interface, so any module can be updated, tested or replaced without affecting the others.
- **Scalability:** The layered, service-oriented architecture allows the application layer to be deployed on cloud infrastructure with load balancing and auto-scaling, so the system can absorb the sharp rise in transactions during major festivals.
- **Maintainability:** Clear separation of presentation, business-logic and data layers, together with documented module APIs and version control, keeps the codebase easy to extend as the temple's needs evolve (e.g., adding new payment modes or regional languages).
- **Reliability:** Automated daily backups, redundant hosting and structured error handling ensure that donation records and event data are never lost, even during peak-load periods.
- **Usability:** A simple, multilingual interface (e.g., Tamil, Telugu, Hindi, English) and accessible design accommodate devotees and staff with varying levels of digital literacy, including elderly volunteers.
- **Security:** End-to-end encryption, PCI-DSS-compliant payment handling, strict role-based access control and an immutable audit log protect sensitive donor and financial data and deter fraud.

5. Justify the Chosen Methodology/Framework

5.1 Selected Methodology: Agile (Scrum)

The Agile Scrum framework is recommended for developing the Temple Donation and Event Management System. The trust board (acting as Product Owner) maintains a prioritised product backlog; the development team works in fixed sprints of 2–3 weeks, each ending with a working, demonstrable increment; and daily stand-ups, sprint reviews and retrospectives keep the team, trustees and key volunteers continuously aligned.

5.2 Comparison with Other Methodologies

Methodology	Advantage	Why Not Ideal Here	Verdict
Waterfall	Simple, well-documented	Requirements are unlikely to be fully known upfront; no room for stakeholder feedback until the end — risky for a first-of-its-kind temple system.	Not suitable
Spiral	Good for risk-heavy, large projects	Adds process overhead unnecessary for a moderately sized system with a small in-house/vendor team.	Not suitable
Scrum (Agile)	Iterative delivery, continuous trustee/donor feedback, early MVP, adapts to changing regulations	Requires disciplined sprint planning and an available Product Owner (trust board representative).	Recommended

5.3 Justification

- Temple committees often refine their requirements as they see the system in action (e.g., deciding mid-way to add a specific report format for the Endowment Department) — Scrum's iterative cycle accommodates this naturally, whereas a Waterfall approach would require re-starting the requirements phase.
- An early, minimal version — donation entry with instant receipts — can be released within the first few sprints and used in real temple operations, delivering value quickly while the event-management and reporting modules are still being built.
- Regular sprint reviews with trustees, accountants and volunteer coordinators ensure the system genuinely fits real temple workflows rather than developers' assumptions, reducing costly rework later.
- Festival dates are fixed and non-negotiable; Scrum's short, time-boxed sprints make it easier to prioritise and deliver festival-critical features (e.g., hall-booking conflict checks) ahead of a known deadline.
- Once live, a DevOps-style continuous-integration/continuous-deployment pipeline can be layered onto the Scrum process for rapid bug fixes and incremental feature releases (e.g., scaling up before a major festival) without disrupting ongoing temple operations.

5.4 Supporting Successful Development and Maintenance

Because Scrum keeps stakeholders involved throughout, the delivered system is more likely to match actual temple operations from day one, reducing costly post-launch changes. After go-live, the same sprint cadence is reused for a maintenance backlog — bug fixes, regulatory-format changes to receipts/certificates, and seasonal scaling — so the system continues to evolve safely over its lifetime rather than becoming a rigid, one-time deliverable.

6. Discuss Expected Outcomes and Limitations

6.1 Expected Benefits

Benefit	How It Is Achieved
Transparency & Trust	Real-time dashboards and digital audit trails let trustees and devotees see exactly how funds are collected and used.
Wider Donor Reach	Online payment options let NRIs and remote devotees contribute easily, increasing overall donations.
Faster Service	Instant digital receipts and 80G certificates remove queues and manual paperwork at the counter.
Efficient Event Management	Automated conflict detection prevents double-booking of halls, priests, and resources.
Reduced Fraud Risk	Role-based access control and immutable logs make misappropriation of funds far harder to conceal.
Easier Compliance	One-click report exports simplify submissions to the Endowment Department and Income-Tax authorities.

6.2 How the Solution Addresses the Identified Challenges

Automated receipting and donor history directly resolve devotees' complaints about lack of transparency; conflict-checking in the event module removes the double-booking problem faced by event coordinators; role-based dashboards give the trust board the real-time financial visibility they previously lacked; and the immutable audit trail combined with RBAC substantially reduces the fraud and reconciliation burden currently placed on accountants.

6.3 Potential Risks and Implementation Challenges

- Resistance to change from long-serving staff and elderly volunteers accustomed to paper-based processes.
- Digital-literacy gaps among some devotees, particularly in rural areas, limiting adoption of online donation channels.
- Upfront cost of hardware, software licensing/hosting and staff training during the digital transition.
- Data-security and privacy concerns around storing donor personal and financial information.
- Dependence on stable internet connectivity and power supply, which can be unreliable in some temple locations.
- Sudden traffic spikes during major festivals could strain the system if scaling is not properly planned and load-tested in advance.
- Endowment/regulatory requirements vary between states, requiring the reporting module to remain configurable.

6.4 Limitations of the Proposed Solution

The system cannot fully replace in-person cash handling for devotees who prefer or can only make traditional offerings, so a parallel manual-entry option must be retained. It also depends on continuous internet/power availability for real-time features, and its effectiveness depends on ongoing training and change-management support for staff and volunteers, not just the technology itself.

6.5 Suggested Future Enhancements

- AI-based anomaly detection to flag unusual donation or reconciliation patterns for early fraud prevention.
- A devotee-facing chatbot to answer FAQs, check donation status, and assist with event bookings.
- Live-streaming integration for poojas and festivals for devotees who cannot attend in person.
- A blockchain-backed ledger for donations to further strengthen transparency and public trust.
- A dedicated mobile app with voice support in regional languages for greater accessibility.

- Integration with government single-window compliance portals to further automate statutory reporting.

Conclusion

The Temple Donation and Event Management System replaces a fragmented, manual, paper-based process with an integrated, modular, and secure digital platform. By applying core software engineering principles — modularity, scalability, maintainability, reliability, usability and security — and adopting an Agile Scrum development approach, the proposed solution can be delivered incrementally, validated continuously with real stakeholders, and evolved safely over time. The result is a system that improves transparency and trust for devotees, streamlines operations for temple staff and volunteers, and gives the trust board the real-time oversight needed for sound governance and statutory compliance.